Nicheflow - Project Proposal

Team Name: Group 6

Team Members: Madison Allen, Roddy McFarlane, Isaac Alawiye, Oluwafunto Odunaike

1. Target Project & Scope

NicheFlow is a marketplace for short-term rentals, booking services and booking places within the user's local area. Streamlining the process of renting and booking for both sides by having everything needed in order to book and/or rent happen on the same site. Customer users can search for what they need in their local area and when they find a listing that matches what they are looking for they can book/rent directly on the site picking their day/s and/or times, complete the payment and listing provided instruction on how to pick up or access their rental/booking. While listing users can create new listings, block certain days or times when their listing is unavailable, upload photos to show off the listing, provide a description and even provide instructions that are only shown to booked customers. After the rental/booking both types of users can rate each other in order to allow for other users to gauge reputation and to give feedback to each other. The site also allows for administrators to review listings and reviews, and remove any that may violate any terms and conditions and/or laws.

- **Problem Statement & Objectives:**

This web application attempts to help people looking to rent to people offering rentals, and similarly to bookings. Making the site a one-stop shop for any and all renters' needs while also making the process of offering rentals easier to do, streamlining the related bookkeeping activities required to offer a rental/booking.

- **Key Features & Functionality:**

NicheFlow is organized into four modules; Authentication & Accounts (AUTH), Listings & Search (LST), Booking & Payment (BK), and Admin & Reviews (ADM). Features are prioritized as follows: **M** = in the October 25 MVP, **S** = confident for the final version, **C** = desired, but we are not yet sure how to implement it. Each feature has an ID so it can be traced to automated tests in the Living Specification.

Must Have (October MVP)

ID	Feature	Capability and testing focus
AUTH-01	Registration and login	Email and password sign-up with secure password hashing and session or token management. Tests cover invalid credentials, expired sessions, and duplicate accounts.

AUTH-02	Role-based access control	Customer, supplier, and admin roles. Tests cover privilege escalation attempts (e.g., a customer calling supplier or admin endpoints).
LST-01	Supplier listing management	Suppliers create, edit, and deactivate listings (title, category, price, availability slots). Tests cover input validation and ownership checks.
LST-02	Browse and basic search	Keyword and category search with price filters. Tests cover empty results, special characters, and pagination boundaries.
BK-01	Booking with availability check	Customers book an open slot. The system blocks double-booking, so if two customers book the last slot at once, exactly one succeeds. Tests include concurrent requests.
BK-02	Booking state machine	Lifecycle of pending → confirmed → completed, plus cancelled. Tests cover every valid transition and reject every invalid one.
BK-03	Payment through a mocked Stripe layer	Charge on booking, with idempotency keys so retries never double-charge. The mock layer will be WireMock or an equivalent.
BK-04	Cancellation and refund calculation	The refund amount depends on cancellation timing. Unit tests cover the calculation rules, including boundary cases.
BK-05	Timezone-safe scheduling	Times are stored in UTC and displayed in the viewer's timezone. Tests cover DST changes and mismatched customer and supplier timezones.
ADM-01	Basic admin console	Admins view users and bookings and can cancel a booking.
QA-01	Test and CI baseline	Unit and integration tests run automatically on every pull request through GitHub Actions.

Should Have (final version)

ID	Feature	Capability and testing focus
BK-06	Dispute workflow	Adds the disputed state, with an admin resolution flow and a full audit trail.

BK-07	Webhook handling and retries	Handles the case where a Stripe charge succeeds but the webhook times out. Retries are consistent.
BK-08	Concurrent cancellation safety	If an admin and a supplier cancel the same booking, the refund is issued exactly once.
ADM-02	Reviews and ratings	Reviews are allowed only after a completed booking, one per booking, with timing validation.
ADM-03	Moderation tools	Admins can hide listings and reviews.
LST-03	Advanced search	Sorting, date-availability filtering, and search that copes with stale data.
BK-09	Discount rules and promo codes	Unit-testable pricing logic, including stacking rules.
NOT-01	Email notification queue	Booking confirmations and cancellations sent asynchronously, with tests for queue failure and retry.
QA-02	Performance baseline and reporting	Benchmarks for concurrent booking and search response times, with Allure reports and a BDD (Cucumber) suite.

Could Have (desired, implementation uncertain)

ID	Feature	Why it is uncertain
NOT-02	Real-time WebSocket notifications	Adds infrastructure complexity and is hard to test deterministically.
ADM-04	Review fraud detection	Needs a defensible detection heuristic (e.g., burst patterns or account-age signals).
BK-10	Multi-currency support with conversion	Requires exchange-rate sourcing and rounding rules that never lose money.
LST-04	Eventually consistent search index	A separate search index means handling sync lag deliberately.
BK-11	Supplier payouts and calendar sync	Real payout flows and iCal or Google Calendar integration go beyond a sandbox.

Out of scope**:** native mobile apps and live payment processing (Stripe stays in test mode).

Tooling Strategy

Because this course allows us to build with AI agents and validate what they produce, our strategy separates generating code from verifying it.

AI development tools

- GitHub Copilot, OpenCode, and Claude code would be utilized for inline completion in the IDE, and first-pass pull request review as a second opinion, never as a replacement for human review.

Quality and verification tools

Our stack is Java (Spring Boot) on the backend, and React on the frontend, so verification tooling is chosen to cover both sides rather than hedging between languages.

- Coverage: JaCoCo for the Java backend and Jest's built-in coverage reporting for the React frontend, both checked against our 80% target.
- Static analysis: SpotBugs and Checkstyle for Java, ESLint and Prettier for JavaScript and React.
- Mutation testing: PIT for Java and Stryker Mutator (StrykerJS) for JavaScript, so we can check that tests actually catch injected bugs on both the backend and the frontend, and don't just pass.
- Test infrastructure: Playwright for E2E, React Testing Library for component tests, Testcontainers for database isolation, WireMock for Stripe, Cucumber for BDD, and Allure for reporting.

Working rules to control agent risk

1. Spec first. Each feature starts as a requirement ID in the Living Specification, and agents generate code and tests against it.
2. Small, reviewable pull requests. A different team member than the prompt author reviews every AI-generated diff.
3. Human-owned edge cases. We write the high-risk tests ourselves, or review them line by line: double-booking, ensuring consistency, refund correctness, and RBAC.
4. Shared project rules file (e.g., CLAUDE.md) that records architecture and naming conventions, to limit architectural drift across agent sessions.
5. Failure log. We record hallucinated APIs (especially Stripe), broken logic, and trivially passing tests, and use the log for the weekly reflections and the final presentation on agent productivity versus technical debt.

2. Targeted Platform & Initial Programming Language / Libraries Used

Target Platform:

NicheFlow is a full-stack web application composed of three primary layers: a React-based single-page frontend, a RESTful backend API built with Java (Spring Boot), and a PostgreSQL relational database. The frontend and backend are decoupled and communicate over HTTPS,

with WebSocket support (native to Spring Boot) layered on top for real-time features such as booking status updates and notifications. The application is accessed through standard web browsers on both desktop and mobile devices; no native mobile or desktop application is planned for this scope.

Programming Language(s):

- Backend: Java, using a framework called Spring Boot. Java is a good fit here because it's reliable for handling important logic like payments and bookings without errors, and it has strong, well-tested tools for automatically testing our code — which matters a lot for this project since testing is a core part of what we're building.
- Frontend: JavaScript, using React (a popular tool for building interactive websites), along with HTML and CSS for layout and styling.
- Database Access: We will use a tool called JPA/Hibernate, which lets our Java code talk to the database without needing to write raw database queries by hand.

Frameworks & Libraries: Frameworks, UI libraries, or backend databases planned.

WHAT'S IT FOR	**TOOL WE'RE USING**	**IN PLAIN TERMS**
Backend Framework	Spring Boot	The foundation of our server; handles requests coming from the website
Talking to the database	Spring Data JPA / Hibernate	Lets our Java code save and load data without writing raw SQL
Front-end Framework	React	Builds the actual website pages users interact with
Database	PostgreSQL	Where all our data lives; user accounts, listings, bookings, payments, reviews
Payments	Stripe (using a test/mock version)	Handles payment processing; we'll use a fake version for testing so no real money moves during development
Real-time updates	WebSockets	Lets the app instantly notify users (e.g., "your booking is confirmed") without refreshing
Backend testing	JUnit	Checks that individual pieces of our backend code work correctly

Full user flow testing	Playwright	Simulates a real user clicking through the whole site, start to finish
Behavior-based testing	Cucumber	Lets us write test cases in plain English so anyone on the team can read them
Automation pipeline	GitHub Actions	Automatically runs our tests every time we update the code
Test result reports	Allure Reports	Turns our test results into an easy-to-read report/dashboard
Isolated test databases	Testcontainers	Creates a temporary, throwaway database just for running tests, so we don't mess up real data

3. Phase 1: Proposal → Midterm Prototype

Goal: Core features (auth, listings, booking, payment mock)

Week	Dates	Focus Area	Key Deliverables
1	Sept 18–21	Proposal & Architecture	Scope, tech stack, repo setup, Proposal submission
2	Sept 22–28	Project Setup	Backend + frontend scaffolding, DB schema, CI/CD skeleton
3	Sept 29–Oct 5	Authentication	User accounts, auth UI, unit tests
4	Oct 6–12	Listings & Search	Listings CRUD, search API, listing UI, integration tests
5	Oct 13–19	Booking MVP	Booking lifecycle, payment mock, booking UI, tests
6	Oct 20–25	MVP Polish	Reviews, notifications, E2E tests, Midterm demo

Phase 2: Post-Midterm → End of Term

Goal: Advanced logic, concurrency, full test suite, performance, UI polish

Week	Dates	Focus Area	Key Deliverables
7	Oct 26–Nov 1	Advanced Booking Logic	Concurrency fixes, refunds, idempotent payments
8	Nov 2–8	Admin & Reviews	Admin dashboard, review fraud detection
9	Nov 9–15	Performance & Security	Load tests, RBAC security tests, timezone/currency integrity
10	Nov 16–22	Full E2E Coverage	Playwright suite, Allure reports, CI/CD regression
11	Nov 23–29	UI Polish & Spec Updates	UX improvements, WebSockets, Living Spec updates
12	Nov 30–Dec 5	Final Presentation Prep	Demo script, architecture diagrams, test coverage summary

Team Roles & Responsibilities:

Role	Member	Responsibilities
Backend Lead	Isaac	API architecture, auth, bookings, payments, DB schema, unit/integration tests
Frontend Lead	Madison	React UI, listings, booking flow, component tests, Playwright E2E
QA/Test Automation Lead	OluwafuntoFunto	Test strategy, BDD, unit/integration/E2E coverage, performance/security tests, Allure
DevOps Lead	Roddy	CI/CD pipeline, PR workflow, regression suite, code quality tools